

Residential Society Management & Maintenance Billing System

DBMS Course Project Report

Student Name	Sri Pranav Ramini
Roll Number	25WU0102273
Course	Database Management Systems (DBMS)
Academic Year	2025–2029
Project Type	DBMS Course Project
University	Woxsen University
Branch	CSE - AIML

Table of Contents

1. Abstract	3
2. Introduction	3
3. Problem Statement	3
4. Objectives and Scope	3
5. ER Diagram	4
5.1 Explanation of the ER Diagram	4
6. Relational Schema · Separate Table for Each Entity.....	5
6.1 SOCIETY	5
6.2 FLAT	5
6.3 RESIDENT	5
6.4 COMPLAINT	5
6.5 MAINTENANCE_TYPE	6
6.6 MAINTENANCE_BILL	6
6.7 PAYMENT	6
6.8 BILL_DETAIL	7
7. Database and Table Creation	7
8. Sample SQL Queries and Outputs	7
8.1 Complaint Count by Status	7
8.2 Total Maintenance Amount	8
8.3 Number of Residents in Each Flat	8
9. User Interface Screenshots	10
9.1 Dashboard	10
9.2 Payments Module	10
9.3 Flats Module	11
10. Implementation Details	11
11. Testing and Results	11
12. Conclusion	11
13. Future Enhancements	11
14. GitHub Repository	11

1. Abstract

The Residential Society Management & Maintenance Billing System is a database management project developed to organize and manage the major activities of a residential society through a centralized relational database. The system models information about societies, flats, residents, complaints, maintenance types, maintenance bills, payments, and bill details. Instead of keeping these records independently, the project connects them through primary keys and foreign keys so that related information can be accessed consistently.

The database is implemented in MySQL and is connected to a web-based interface called ResidentCare. The interface provides separate modules for viewing and managing society records, flats, residents, complaints, maintenance types, maintenance bills, payments, and bill details. It also provides dashboard statistics and useful SQL-based reports. The system demonstrates practical database concepts such as entity-relationship modelling, relational schema design, constraints, joins, aggregation, CRUD operations, and referential integrity.

The ER diagram forms the foundation of the project. It identifies eight main entities and the relationships between them. The design helps reduce duplication of data and makes it possible to trace information from a society to its flats and residents, from flats to complaints and bills, and from bills to payments and detailed maintenance components. The final implementation therefore combines database design with a simple user interface to demonstrate how a real-world residential society management system can be organized using DBMS principles.

2. Introduction

Residential societies generate a large amount of information every day. A society contains multiple flats, each flat may have one or more residents, residents can raise complaints, and maintenance bills have to be generated, tracked, and paid. Maintenance charges can also consist of different maintenance types such as common-area services, water-related services, security, or other society activities. Managing these records manually can lead to duplication, inconsistent information, and difficulty in retrieving related records.

A relational database provides a structured solution to this problem. In this project, each major real-world object is represented as an entity and then converted into a relational table. Primary keys uniquely identify records, while foreign keys connect related tables. For example, a flat contains a Society_ID to identify its society, a resident contains a Flat_ID to identify the flat occupied, and a maintenance bill contains a Flat_ID to identify the flat for which the bill was generated.

The project follows the ER diagram shown below as the main database design. The relationships represented in the diagram are used to create the MySQL tables and to write SQL queries for reporting and data management. The web interface then uses the same database to display records and perform operations such as creating, updating, deleting, viewing, searching, and filtering data.

3. Problem Statement

The objective is to design and implement a database system that can efficiently manage residential society information, apartment records, residents, complaints, maintenance billing, payment details, and maintenance components while maintaining proper relationships and data integrity.

4. Objectives and Scope

- Design an ER model for the residential society management system.
- Convert the ER model into normalized relational tables.
- Implement the database using MySQL.
- Maintain primary-key and foreign-key relationships.
- Support CRUD operations through the application interface.
- Track maintenance bills, payment modes, and payment transactions.
- Record complaints and their status.
- Generate useful reports using SQL functions, GROUP BY and JOIN operations.

5. ER Diagram

The following ER diagram is the central design of the project. It contains eight entities: SOCIETY, FLAT, RESIDENT, COMPLAINT, MAINTENANCE BILL, PAYMENT, BILL DETAIL, and MAINTENANCE TYPE. The diagram also shows the primary keys, foreign keys, attributes, and cardinalities of the relationships.

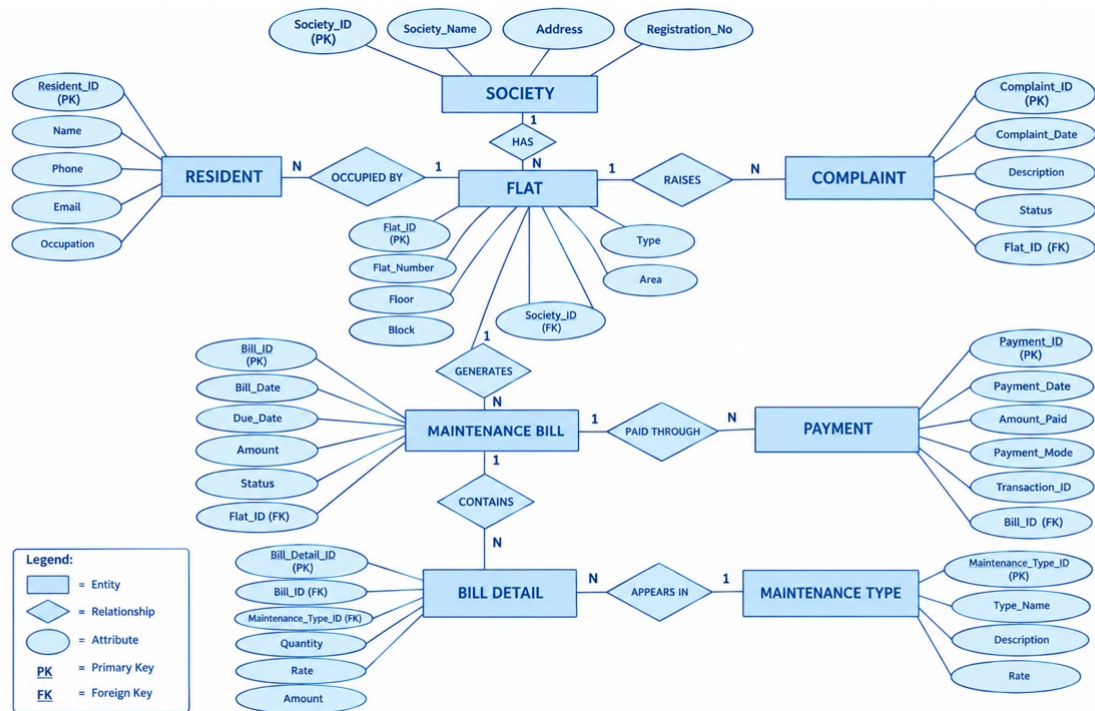


Figure 1. ER diagram of the Residential Society Management & Maintenance Billing System.

5.1 Explanation of the ER Diagram

- SOCIETY has multiple FLAT records. The relationship is represented as 1:N.
- A FLAT can be occupied by multiple RESIDENT records, while each resident is linked to one flat in the given design.
- A FLAT can raise multiple COMPLAINT records.
- A FLAT can generate multiple MAINTENANCE BILL records.
- A MAINTENANCE BILL can have multiple PAYMENT records through the PAID THROUGH relationship.
- A MAINTENANCE BILL contains multiple BILL DETAIL records.
- Each BILL DETAIL record refers to a MAINTENANCE TYPE, allowing a bill to be broken into individual maintenance components.

6. Relational Schema · Separate Table for Each Entity

Each entity from the ER diagram is implemented as a separate relational table. The following sections intentionally provide one independent schema table for every entity.

6.1 SOCIETY

Stores residential society information.

Attribute	Data Type	Key / Constraint
Society_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Society_Name	VARCHAR(100)	NOT NULL
Address	VARCHAR(255)	NOT NULL
Registration_No	VARCHAR(50)	NOT NULL, UNIQUE

6.2 FLAT

Stores apartment information and its society relationship.

Attribute	Data Type	Key / Constraint
Flat_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Flat_Number	VARCHAR(20)	NOT NULL
Floor	INT	NOT NULL
Block	VARCHAR(10)	NOT NULL
Type	VARCHAR(20)	NOT NULL
Area	DECIMAL(10,2)	NOT NULL
Society_ID	INT	FOREIGN KEY → SOCIETY(Society_ID)

6.3 RESIDENT

Stores resident information and the flat occupied.

Attribute	Data Type	Key / Constraint
Resident_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Name	VARCHAR(100)	NOT NULL
Phone	VARCHAR(15)	NOT NULL
Email	VARCHAR(100)	NULL
Occupation	VARCHAR(100)	NULL
Flat_ID	INT	FOREIGN KEY → FLAT(Flat_ID)

6.4 COMPLAINT

Stores complaints and their current status.

Attribute	Data Type	Key / Constraint
Complaint_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Complaint_Date	DATE	NOT NULL
Description	VARCHAR(255)	NOT NULL
Status	ENUM('Pending','In Progress','Resolved')	NOT NULL
Flat_ID	INT	FOREIGN KEY → FLAT(Flat_ID)

6.5 MAINTENANCE_TYPE

Stores maintenance categories and their rates.

Attribute	Data Type	Key / Constraint
Maintenance_Type_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Type_Name	VARCHAR(100)	NOT NULL
Description	VARCHAR(255)	NULL
Rate	DECIMAL(10,2)	NOT NULL

6.6 MAINTENANCE_BILL

Stores maintenance bills generated for flats.

Attribute	Data Type	Key / Constraint
Bill_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Bill_Date	DATE	NOT NULL
Due_Date	DATE	NOT NULL
Amount	DECIMAL(10,2)	NOT NULL
Status	ENUM('Paid','Pending')	NOT NULL
Flat_ID	INT	FOREIGN KEY → FLAT(Flat_ID)

6.7 PAYMENT

Stores payments made against maintenance bills.

Attribute	Data Type	Key / Constraint
Payment_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Payment_Date	DATE	NOT NULL
Amount_Paid	DECIMAL(10,2)	NOT NULL

Attribute	Data Type	Key / Constraint
Payment_Mode	ENUM('UPI','NEFT','Card','Cash')	NOT NULL
Transaction_ID	VARCHAR(50)	NOT NULL, UNIQUE
Bill_ID	INT	FOREIGN KEY → MAINTENANCE_BILL(Bill_ID)

6.8 BILL_DETAIL

Stores individual maintenance components included in a bill.

Attribute	Data Type	Key / Constraint
Bill_Detail_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Bill_ID	INT	FOREIGN KEY → MAINTENANCE_BILL(Bill_ID)
Maintenance_Type_ID	INT	FOREIGN KEY → MAINTENANCE_TYPE(Maintenance_Type)
Quantity	INT	NOT NULL
Rate	DECIMAL(10,2)	NOT NULL
Amount	DECIMAL(10,2)	NOT NULL

7. Database and Table Creation

The database is implemented in MySQL using the relational schema derived from the ER diagram. Tables are created in dependency order so that referenced parent tables exist before foreign-key relationships are added. Sample data is then inserted into the tables for testing and demonstration.

8. Sample SQL Queries and Outputs

8.1 Complaint Count by Status

```
SELECT Status, COUNT(*) AS Number_of_Complaints
FROM COMPLAINT
GROUP BY Status;
```

```
mysql>
mysql> SELECT
      ->     Status,
      ->     COUNT(*) AS Number_of_Complaints
      -> FROM COMPLAINT
      -> GROUP BY Status;
```

Status	Number_of_Complaints
Resolved	12
Pending	7
In Progress	6

```
3 rows in set (0.006 sec)

mysql> |
```

Figure 2. MySQL output for complaint count by status.

8.2 Total Maintenance Amount

```
SELECT SUM(Amount) AS Total_Maintenance_Amount
FROM MAINTENANCE_BILL;
```

```
mysql>
mysql>
mysql> -- Query 7: Total maintenance amount
Query OK, 0 rows affected (0.002 sec)

mysql>
mysql> SELECT
      ->     SUM(Amount) AS Total_Maintenance_Amount
      -> FROM MAINTENANCE_BILL;
```

Total_Maintenance_Amount
95200.00

```
1 row in set (0.009 sec)
```

Figure 3. MySQL output for total maintenance amount.

8.3 Number of Residents in Each Flat

```
SELECT F.Flat_Number, COUNT(R.Resident_ID) AS Number_of_Residents
FROM FLAT F
LEFT JOIN RESIDENT R ON F.Flat_ID = R.Flat_ID
GROUP BY F.Flat_ID, F.Flat_Number;
```

```

mysql>
mysql> SELECT
->     F.Flat_Number,
->     COUNT(R.Resident_ID) AS Number_o
-> FROM FLAT F
-> LEFT JOIN RESIDENT R
-> ON F.Flat_ID = R.Flat_ID
-> GROUP BY F.Flat_ID, F.Flat_Number;
+-----+-----+
| Flat_Number | Number_of_Residents |
+-----+-----+
| A101        | 1                    |
| A102        | 1                    |
| A201        | 1                    |
| A202        | 1                    |
| B101        | 1                    |
| B102        | 1                    |
| B201        | 1                    |
| B202        | 1                    |
| C101        | 1                    |
| C102        | 1                    |
| C201        | 1                    |
| C202        | 1                    |
| D101        | 1                    |
| D102        | 1                    |
| D201        | 1                    |
| D202        | 1                    |
| E101        | 1                    |
| E102        | 1                    |
| E201        | 1                    |
| E202        | 1                    |
| F101        | 1                    |
| F102        | 1                    |
| F201        | 1                    |
| F202        | 1                    |
| G101        | 1                    |
+-----+-----+
25 rows in set (0.400 sec)

```

Figure 4. MySQL output for residents per flat.

9. User Interface Screenshots

The ResidentCare interface provides a simple frontend for interacting with the MySQL database. The screenshots demonstrate the dashboard, payments module, and flats module. These screens support the practical implementation of the relational database design.

9.1 Dashboard

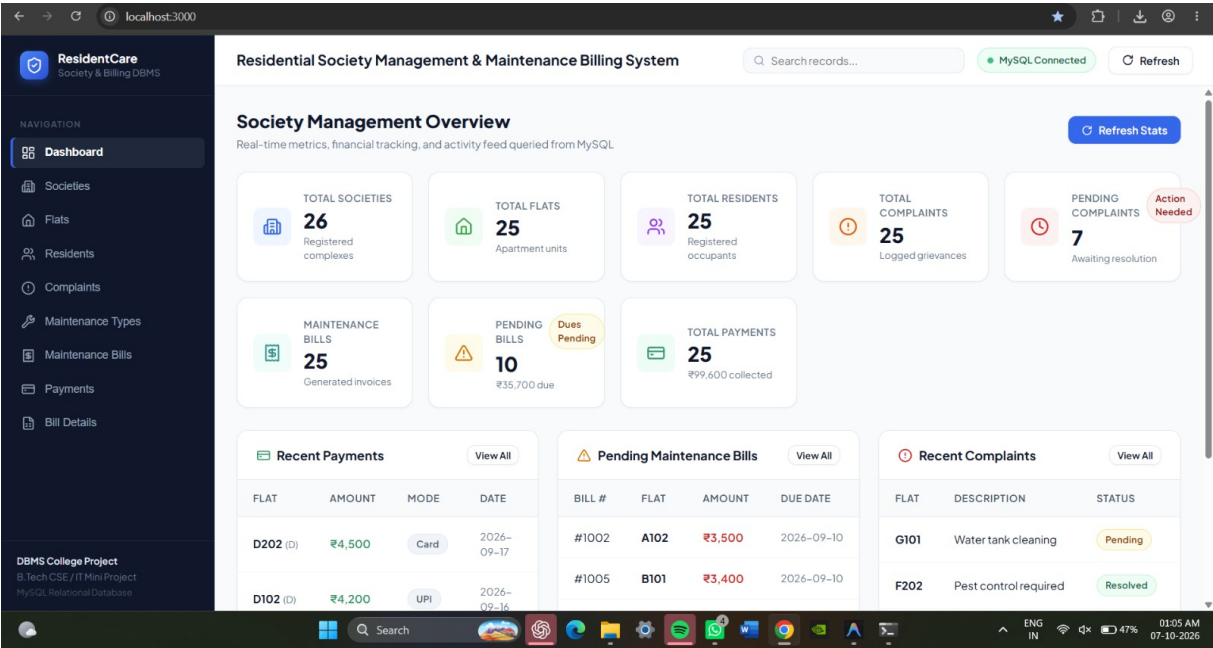


Figure 5. ResidentCare dashboard showing MySQL Connected status.

9.2 Payments Module

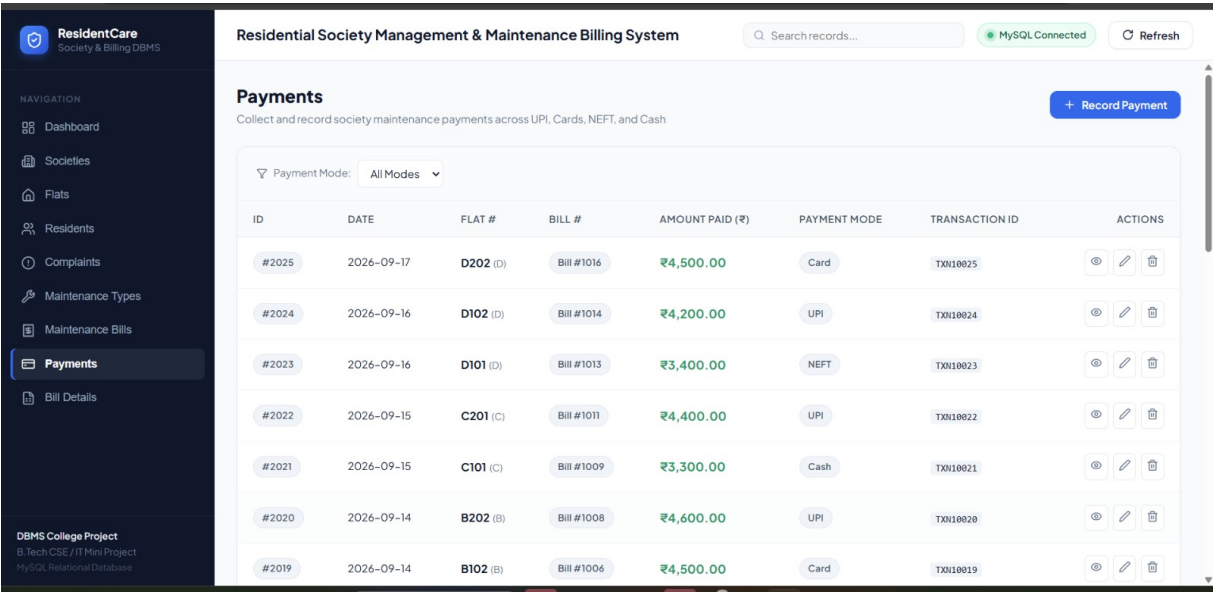
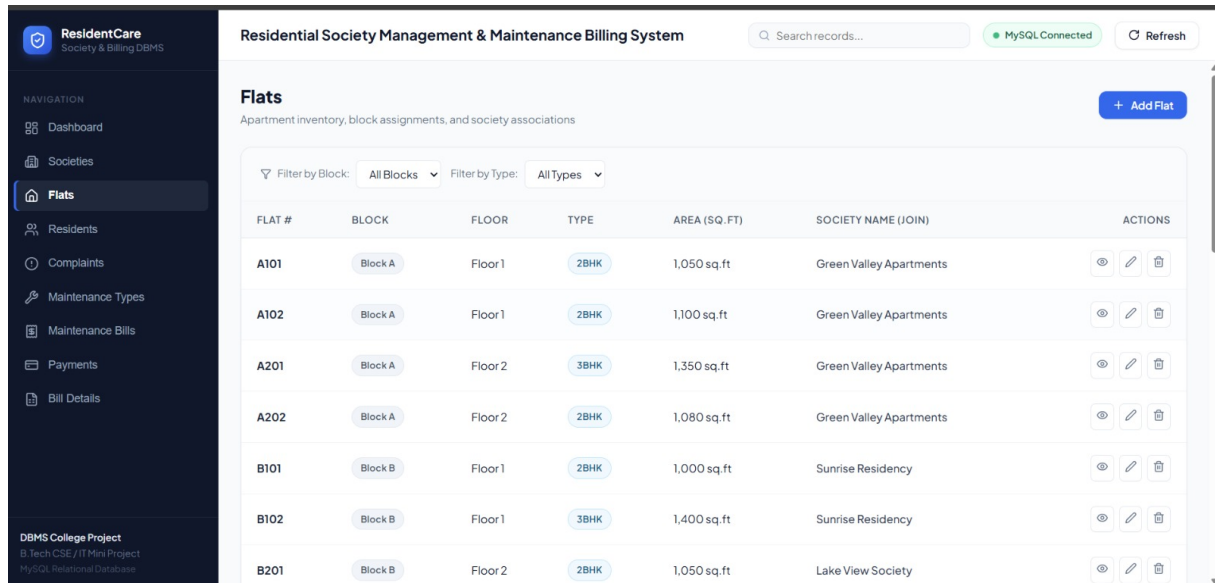


Figure 6. Payments module with records and CRUD actions.

9.3 Flats Module



The screenshot displays the 'ResidentCare Society & Billing DBMS' interface. The left sidebar contains navigation links: Dashboard, Societies, Flats (selected), Residents, Complaints, Maintenance Types, Maintenance Bills, Payments, and Bill Details. The main header shows 'Residential Society Management & Maintenance Billing System' with a search bar, 'MySQL Connected' status, and a 'Refresh' button. The 'Flats' section title is followed by a subtitle 'Apartment inventory, block assignments, and society associations' and an 'Add Flat' button. Below this, there are filters for 'Filter by Block' (All Blocks) and 'Filter by Type' (All Types). The main table lists flats with columns: FLAT #, BLOCK, FLOOR, TYPE, AREA (SQ. FT), SOCIETY NAME (JOIN), and ACTIONS. The table contains 9 rows of data.

FLAT #	BLOCK	FLOOR	TYPE	AREA (SQ. FT)	SOCIETY NAME (JOIN)	ACTIONS
A101	Block A	Floor1	2BHK	1,050 sq.ft	Green Valley Apartments	[Icons]
A102	Block A	Floor1	2BHK	1,100 sq.ft	Green Valley Apartments	[Icons]
A201	Block A	Floor2	3BHK	1,350 sq.ft	Green Valley Apartments	[Icons]
A202	Block A	Floor2	2BHK	1,080 sq.ft	Green Valley Apartments	[Icons]
B101	Block B	Floor1	2BHK	1,000 sq.ft	Sunrise Residency	[Icons]
B102	Block B	Floor1	3BHK	1,400 sq.ft	Sunrise Residency	[Icons]
B201	Block B	Floor2	2BHK	1,050 sq.ft	Lake View Society	[Icons]

Figure 7. Flats module showing society JOIN results.

10. Implementation Details

The application provides navigation for Dashboard, Societies, Flats, Residents, Complaints, Maintenance Types, Maintenance Bills, Payments, and Bill Details. The interface retrieves data from MySQL and presents it in structured tables. The project also demonstrates search, filtering, record viewing, editing, and deletion operations.

11. Testing and Results

The database connection was demonstrated through the MySQL Connected indicator in the interface. The supplied SQL outputs demonstrate aggregation and grouping queries. The complaint query groups records by status, the maintenance query calculates the total bill amount, and the resident query uses a LEFT JOIN and GROUP BY to count residents associated with each flat. The supplied screenshot for the maintenance total shows ₹95,200.00 and the resident-per-flat query returns 25 rows.

12. Conclusion

The Residential Society Management & Maintenance Billing System successfully demonstrates the complete DBMS workflow from ER modelling to relational schema creation, MySQL implementation, sample data insertion, SQL querying, and web-based database interaction. The ER diagram provides a clear representation of the entities and relationships, while the separate relational tables provide a structured way to store and retrieve information. The project therefore connects theoretical DBMS concepts with a practical application.

13. Future Enhancements

- Add authentication and role-based access control.
- Add automated payment receipts.
- Add overdue-bill notifications.
- Add advanced analytics and downloadable reports.
- Add automated database backup and cloud deployment.

14. GitHub Repository

Project source code: <https://github.com/srirpranav/DBMS-Course-Project/tree/main>

GitHub repository link is intentionally placed only at the end of the report.